

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

PROJECT REPORT

Smart CPU Scheduling Simulator

CPU Scheduling Algorithms and Performance Comparison

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

ABHISHEK KUMAR (24BCE2876)

ASHISH RANJAN (24BCE2154)

Operating Systems Project

https://github.com/AshishRanjan999/Smart-CPU-Scheduling-Simulator.git

Abstract

The Smart CPU Scheduling Simulator is an educational application that demonstrates how an operating system selects processes for execution. The supplied implementation uses C++ and provides a console interface for entering process identifiers, arrival times, burst times and priorities. It supports First Come First Served (FCFS), non-preemptive Shortest Job First (SJF), non-preemptive Priority Scheduling and Round Robin with a user-defined time quantum.

For each selected algorithm, the simulator records execution segments and idle intervals in a Gantt-style timeline. It calculates completion time, turnaround time, waiting time and response time for every process, together with average waiting, turnaround and response times. A comparison option executes all four algorithms on the same workload. Input validation checks process counts, unique identifiers, arrival times, burst times, priorities, menu choices and the Round Robin quantum.

The design separates process data, scheduling decisions, metric calculation and output formatting. Standard C++ vectors store process records and results, while a queue maintains the Round Robin ready list. The simulation uses logical CPU time rather than controlling real operating-system processes. The initial proposal suggested a Python graphical application; the supplied implementation establishes a C++ console version of the scheduling core.

This report documents the design, algorithms, implementation, execution procedure and evaluation method. A reproducible three-process example shows how the same inputs produce different execution orders and average metrics. Its results are analytical traces of the documented scheduling rules, not measured runtime benchmarks. The project helps learners connect scheduling theory with visible execution sequences and understand why algorithm performance depends on the workload.

Keywords: CPU scheduling, FCFS, SJF, Priority Scheduling, Round Robin, C++, Gantt chart, waiting time, turnaround time, response time.

Acknowledgement

We express our gratitude to our Operating Systems faculty for the guidance and opportunity to study CPU scheduling through this project. The concepts covered in the course helped us relate process execution order to waiting time, turnaround time and response time.

We also acknowledge the learning environment and resources provided by Vellore Institute of Technology. Standard operating-systems concepts and the C++ Standard Library provided the foundation for designing the scheduling simulator and documenting its behaviour.

This report brings together the project proposal and the supplied implementation material in a structured presentation of the Smart CPU Scheduling Simulator.

Abhishek Kumar - 24BCE2876
Ashish Ranjan - 24BCE2154

Table of Contents

Section	Page
Abstract	2
Acknowledgement	3
Table of Contents	4
Lists of Figures, Tables and Abbreviations	5
Chapter 1: Introduction	6
Chapter 2: Literature Review and Theoretical Background	7
Chapter 3: System Analysis and Design	8
Chapter 4: Key Concepts and Algorithms	9
Chapter 4: Round Robin and Complexity Analysis	10
Chapter 5: Implementation	11
Chapter 5: Execution Instructions and Screenshot	12
Chapter 6: Testing and Results	13
Chapter 7: Performance Evaluation and Comparative Analysis	14
Chapter 7: Comparison Graph and Interpretation	15
Chapter 8: Source Organisation and Reproducibility	16
Chapter 9: Conclusion and Future Scope	17
References	18
Appendix A: Algorithm Pseudocode	19
Appendix B: Additional Test Cases and Expected Outputs	20

Lists of Figures, Tables and Abbreviations

List of Figures

Figure 1. System architecture and data flow - page 8
Figure 2. Supplied console input and algorithm-selection screenshot - page 12
Figure 3. Average scheduling metrics for the example workload - page 15

List of Tables

Table 1. Comparison of scheduling approaches - page 7
Table 2. Implementation complexity - page 10
Table 3. Data structures and responsibilities - page 11
Table 4. Validation and functional test plan - page 13
Table 5. Example process workload - page 14
Table 6. Per-process results - page 14
Table 7. Average performance comparison - page 15 Table
8. Additional test cases - page 20

List of Abbreviations

Term	Expansion
CPU	Central Processing Unit
OS	Operating System
FCFS	First Come First Served
SJF	Shortest Job First
RR	Round Robin
AT / BT	Arrival Time / Burst Time
CT / TAT	Completion Time / Turnaround Time
WT / RT	Waiting Time / Response Time
PID	Process Identifier
GUI / IDE	Graphical User Interface / Integrated Development Environment

Chapter 1: Introduction

1.1 Background

CPU scheduling determines which ready process receives processor time. When several processes compete for a single CPU, the selection rule affects their completion order, time spent waiting and delay before the first response. The standard algorithms studied in Operating Systems illustrate different trade-offs between simplicity, short-job preference, priority and time sharing.

1.2 Problem Statement

Manual scheduling calculations become difficult when processes arrive at different times or execute in multiple time slices. A small error in the ready queue can change the entire Gantt chart and every

derived metric. Students need a repeatable way to enter a workload, inspect its execution order and compare scheduling policies under the same conditions.

1.3 Motivation

The project turns scheduling formulas into a visible sequence of process execution intervals. By varying arrival times, burst lengths, priorities and the quantum, a learner can explain why the resulting averages change. A compact console program also makes the scheduling logic accessible during a laboratory demonstration.

1.4 Objectives

The objectives are to accept and validate process data; implement FCFS, non-preemptive SJF, non-preemptive Priority and Round Robin; record execution and idle intervals; calculate CT, TAT, WT and RT; and provide comparable average metrics for a common workload.

1.5 Scope

The implemented scope is a single-CPU, single-burst-per-process simulation with integer time values. CPU bursts are known in advance. There are no I/O blocking events or context-switch time costs. The current interface is a C++ console application. Graphical animation, export, aging and additional algorithms remain future extensions.

1.6 Applications

The simulator supports Operating Systems laboratory demonstrations, examination preparation, manual-answer verification and small experiments with scheduling parameters. Its logical timelines illustrate textbook algorithms; they do not measure or replace the host operating system scheduler.

Chapter 2: Literature Review and Theoretical Background

2.1 Existing Approaches

Standard Operating Systems teaching introduces scheduling through definitions, Gantt charts and numerical examples. The project proposal identifies manual calculation as useful for small cases but increasingly error-prone as workloads grow. This simulator automates those calculations while retaining a readable timeline. The review here concerns standard scheduling approaches rather than a benchmark of external software products.

Approach	Selection rule	Main trade-off
FCFS	Earliest arrival among ready jobs	Simple; a long job may delay shorter jobs.
SJF	Smallest burst among ready jobs	Favours short jobs; requires burst information.
Priority	Smallest priority number among ready jobs	Represents importance; no aging is implemented.

Round Robin	FIFO ready queue with fixed quantum	Provides repeated CPU access; depends on quantum.

Table 1. Comparison of scheduling approaches

2.2 Preemptive and Non-preemptive Scheduling

FCFS, SJF and Priority run the selected process until its CPU burst finishes in this implementation. A newly arriving process cannot interrupt it. Round Robin limits each dispatch to the smaller of the remaining burst and the time quantum, placing an unfinished process back in the ready queue.

2.3 Performance Measures

Completion time is the final finish time. Turnaround time is CT - AT; waiting time is TAT - BT; and response time is first start - AT. These waiting-time calculations apply to this CPU-only model. For a non-preemptive single burst, waiting and response times are equal. With Round Robin, a process may wait again after its first execution.

2.4 Fairness and Starvation

Repeated short or high-priority arrivals can postpone other work under SJF or Priority policies. The supplied simulator accepts a finite batch, and no aging mechanism is included. Round Robin rotates unfinished processes through a FIFO queue. Fair CPU access does not imply that every workload has the smallest average waiting time.

Chapter 3: System Analysis and Design

3.1 Requirements Analysis

The simulator requires a positive process count, unique process IDs, non-negative arrival times, positive burst times and priorities of at least one. A smaller priority number means a higher priority. Users select one algorithm or the comparison option; Round Robin additionally requires a positive integer quantum.

The program runs locally with a standard C++ compiler and console. Code::Blocks can provide editing, compilation and execution. No database, browser, server or network connection is required by the scheduling logic. The implementation uses vectors, a queue, sorting, formatted output and exceptions.

3.2 System Architecture

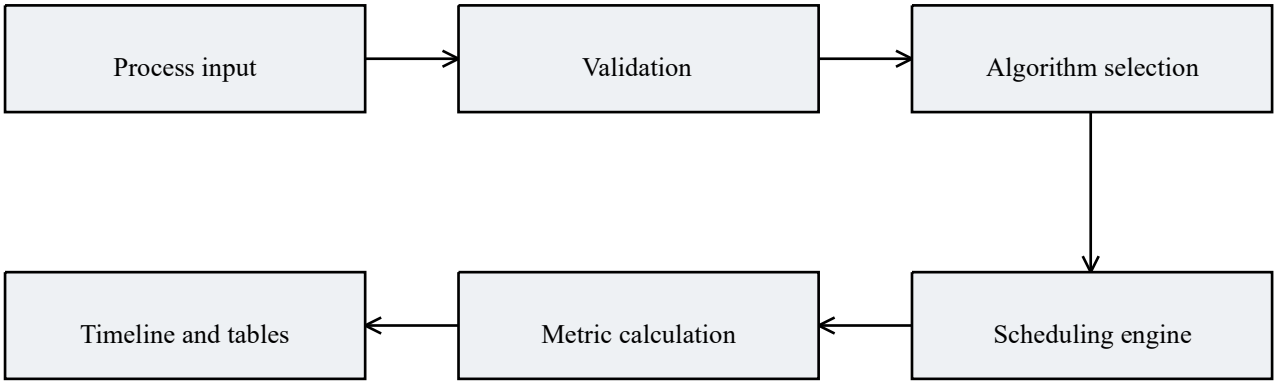


Figure 1. System architecture and data flow

3.3 Modules and Workflow

The input module collects a workload. Validation rejects invalid values before scheduling begins. The engine selects processes and records each start/end interval. The metrics module derives individual and average results. The output module prints the timeline and tables. For Compare All, the engine is called separately for each algorithm using the same process inputs and fresh simulation state.

3.4 Proposal-to-Implementation Change

Review 1 proposed Python, Tkinter and optional Matplotlib charts. The implementation document supplies C++ with a console interface. The scheduling objectives are preserved, while a graphical interface and file export are deferred. The report therefore describes the supplied C++ implementation as the current system.

Chapter 4: Key Concepts and Algorithms

4.1 Shared Simulation State

Each process stores PID, arrival, burst and priority. Simulation state tracks remaining burst, first start, completion and whether the process has finished. The clock begins at zero. A process is eligible only after its arrival. If no process is ready, the clock jumps to the next arrival and an IDLE segment records the gap.

4.2 First Come First Served

The engine selects the earliest-arriving unfinished process from the ready set. Equal arrivals are resolved by lexicographic PID order. The selected process runs for its entire burst. Its first-start and completion times are recorded, then it is marked finished. FCFS is straightforward, but a long process at the front can increase the waiting times of shorter processes behind it.

4.3 Shortest Job First - Non-preemptive

Among ready unfinished processes, SJF selects the smallest original burst time. Ties are resolved first by arrival time and then by lexicographic PID. Once execution starts, later arrivals cannot interrupt the process. This distinction matters: the engine does not sort all jobs by burst and run jobs before they have arrived.

For example, if P1 arrives at time 0 with burst 5 and P2 arrives at time 1 with burst 1, P1 still runs from 0 to 5. P2 starts at 5. Selecting P2 at time 0 would violate its arrival time; interrupting P1 at time 1 would describe a different, preemptive algorithm.

4.4 Priority Scheduling - Non-preemptive

The ready process with the smallest numeric priority is selected. Priority 1 is the highest allowed priority. Equal priorities are resolved by arrival time and then PID. The running job finishes its burst even if a higher-priority job arrives. The implementation does not modify priorities as waiting time increases.

4.5 Tie-breaking and Determinism

The supplied code compares process IDs as strings. Thus P10 sorts before P2 when other selection keys are equal. This is deterministic but different from numeric ID ordering. Demonstrations should use consistent identifiers and document ties so that manual and program results can be compared correctly.

Chapter 4: Round Robin and Complexity Analysis

4.6 Round Robin Scheduling

Round Robin initially orders arrivals by arrival time and then PID. Arrived processes enter a FIFO queue. The engine removes the front process and runs it for min(quantum, remaining burst). The first-start time is set only on its first dispatch. At the end of the slice, newly arrived processes are enqueued before the unfinished current process is re-enqueued.

This ordering includes processes arriving exactly at the slice-end time. If a process completes, its completion time is recorded and it is not re-enqueued. When the queue is empty, the clock advances to the next arrival and records an idle interval. The loop ends after all processes have completed.

4.7 Metric Calculation

```
TAT[i] = completion[i] - arrival[i]
WT[i]  = TAT[i] - burst[i]
RT[i]  = firstStart[i] - arrival[i]
Average WT = sum(WT[i]) / n
Average TAT = sum(TAT[i]) / n
Average RT = sum(RT[i]) / n
```

Totals use floating-point arithmetic for averages. The original burst is retained for metric calculation while Round Robin updates a separate remaining-time array. This prevents partial execution from changing the waiting-time formula.

4.8 Complexity of the Supplied Implementation

Operation	Time complexity	Reason
Validation	$O(n^2)$	Nested duplicate-ID comparisons.
FCFS / SJF / Priority	$O(n^2)$	Scan processes for each selection.
Round Robin core	$O(n \log n + K)$	Sort arrivals; handle K execution slices.
Round Robin incl. validation	$O(n^2 + n \log n + K)$	Duplicate checking also applies.
Storage	$O(n + K)$	State plus timeline; K = sum ceil(BT/q) for RR.

Table 2. Implementation complexity; n is the number of processes

Non-preemptive timelines contain one execution segment per process plus any idle segments. Round Robin may produce many more segments with a small quantum. These bounds describe simulation work, not the cost of scheduling real operating-system threads.

Chapter 5: Implementation

5.1 Tools and Technologies

The supplied program is a C++ console application suitable for a C++11-or-later compiler. It uses iostream for input/output, vector for records and results, queue for Round Robin, algorithm for sorting and minimum selection, iomanip for table formatting, limits for the next-arrival search, and stdexcept for validation errors.

5.2 Data Structures

Structure	Contents and role
Process	PID, arrival time, burst time and priority.
Segment	PID or IDLE label, start time and end time.
Metrics	PID, completion, turnaround, waiting and response time.
Result	Timeline, per-process metrics and three average values.
Working arrays	Remaining bursts, first-start times, completion times and finished flags.

Table 3. Data structures and responsibilities

5.3 Function-level Organisation

validateProcesses: checks the process set and raises an exception for invalid values or duplicate IDs. **simulate:** validates the algorithm, creates fresh working state, constructs a schedule and returns its metrics. **printResult:** prints execution intervals, a metric table and averages. **main:** collects input, presents the menu, selects execution or comparison, and catches exceptions.

5.4 Comparison Option

Menu option 5 calls the simulator once for each supported algorithm. It prints four result sections sequentially, allowing the user to compare the reported averages. This is a console comparison rather than a separate graphical comparison screen. The original process list is passed by constant reference so each run uses the same workload.

5.5 Error Handling and Source Formatting

Invalid numeric extraction and invalid ranges raise runtime errors, which are displayed by the main exception handler. The supplied PDF renders much of its code in uppercase and splits several lines. C++ is case-sensitive: the actual source file must preserve valid keywords, identifiers and newline escapes before compilation. The PDF is documentation, not a reliable copy-and-paste build artifact.

Chapter 5: Execution Instructions and Screenshot

5.6 Execution Procedure

1. Open a C++ Console Application project in Code::Blocks with a configured compiler.
2. Place the correctly formatted implementation in main.cpp and save it.
3. Build and run the project, resolving any compiler messages.
4. Enter the process count, followed by each PID, arrival, burst and priority.
5. Choose FCFS, SJF, Priority, Round Robin or Compare All.
6. Enter a positive quantum if Round Robin or Compare All is selected.
7. Review the timeline, per-process results and average metrics.**5.7 Supplied Application Screenshot**

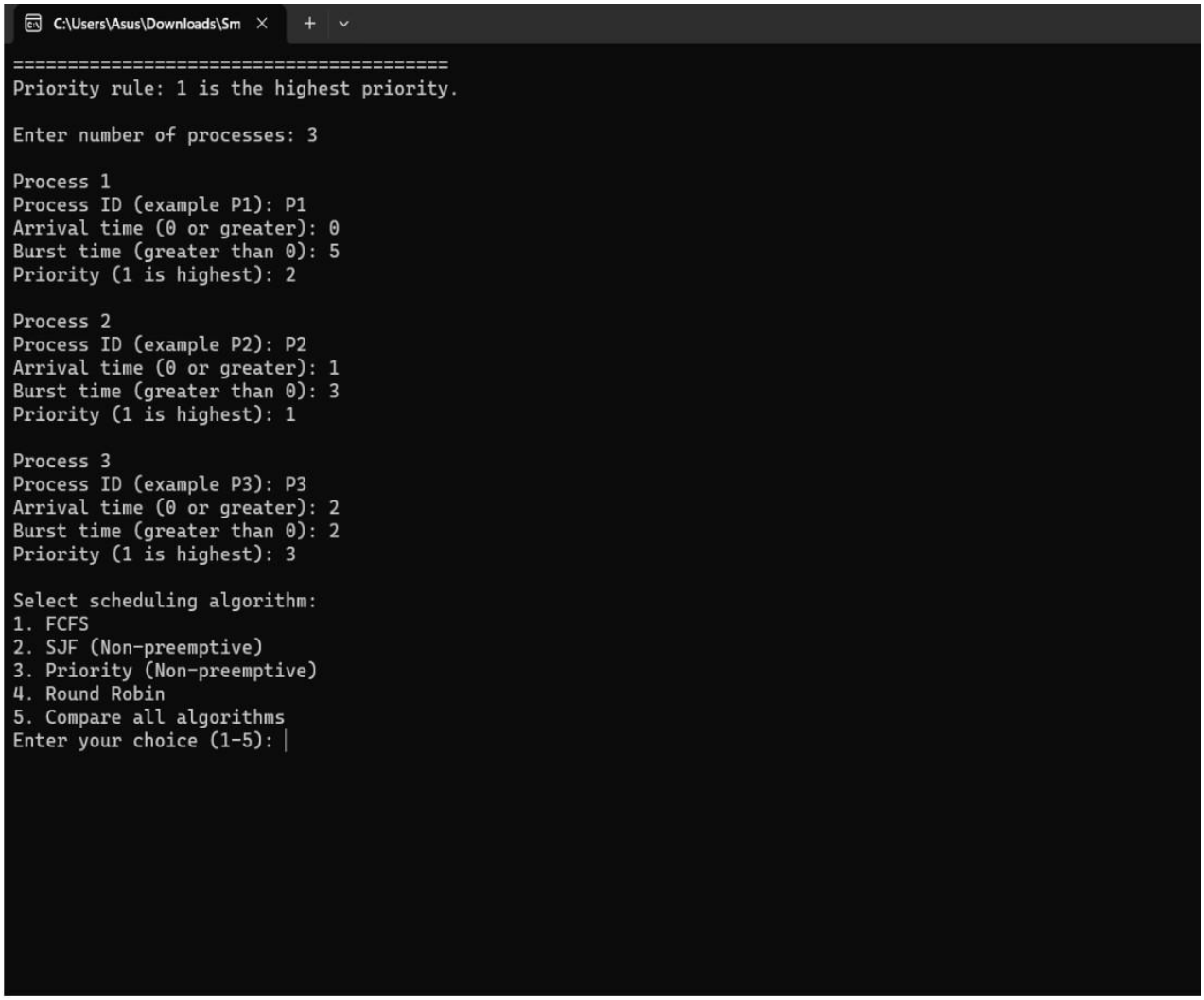


Figure 2. Console input and menu reproduced from the supplied implementation PDF

The screenshot documents entry of three processes and the five-option menu. It does not display completed scheduling results; the evaluation tables that follow are explicitly calculated from the same example inputs.

Chapter 6: Testing and Results

6.1 Testing Strategy

Testing should compare each generated schedule with an independent manual trace. First verify that a process never runs before arrival, that every execution interval has positive length, and that the sum of a process's execution durations equals its burst. Then verify completion and first-start values, followed by the metric formulas.

The supplied evidence contains an input/menu screenshot and code documentation. The cases below are a verification plan and expected outcomes; they are not presented as a completed runtime test log. Numerical results in Chapter 7 are analytical reference values for checking the executable.

Case	Input condition	Expected outcome

T1	Single process	One execution interval; WT = RT = 0 if CPU waits for arrival.
T2	All arrivals equal	Use algorithm key and PID tie-breaks.
T3	Staggered arrivals	Choose only currently ready processes.
T4	No ready process	Record IDLE and jump to next arrival.
T5	Equal burst / priority	Resolve by arrival, then string PID.
T6	RR with short quantum	Multiple slices; total burst preserved.
T7	Arrival at slice boundary	Enqueue new arrival before current unfinished job.
T8	Duplicate PID / invalid range	Reject with an explanatory error.
T9	Invalid menu or non-integer numeric input	Reject; do not run a partial simulation.
T10	Compare All	Reinitialise state for each algorithm.

Table 4. Validation and functional test plan

6.2 Result Consistency Checks

For every process, CT must be at least AT + BT, while WT and RT must be non-negative. RT must not exceed WT in this CPU-only model. For non-preemptive algorithms, WT equals RT. The final clock must account for all busy and idle time. An RR slice must not exceed its quantum, even when the same process is dispatched consecutively.

Chapter 7: Performance Evaluation and Comparative Analysis

7.1 Experimental Workload

The workload below matches the visible process inputs in the supplied screenshot. Time is measured in logical units; context-switch overhead is zero. Round Robin uses q = 2. All values below are calculated traces, not benchmark measurements.

PID	Arrival	Burst	Priority
P1	0	5	2
P2	1	3	1
P3	2	2	3

Table 5. Example process workload

7.2 Execution Timelines

FCFS: P1 [0-5], P2 [5-8], P3 [8-10].
SJF: P1 [0-5], P3 [5-7], P2 [7-10].
Priority: P1 [0-5], P2 [5-8], P3 [8-10].
RR, q = 2: P1 [0-2], P2 [2-4], P3 [4-6], P1 [6-8], P2 [8-9], P1 [9-10]. **7.3**

Per-process Results

Algorithm	PID	CT	TAT	WT	RT
FCFS	P1	5	5	0	0
FCFS	P2	8	7	4	4
FCFS	P3	10	8	6	6
SJF	P1	5	5	0	0
SJF	P2	10	9	6	6
SJF	P3	7	5	3	3
Priority	P1	5	5	0	0
Priority	P2	8	7	4	4
Priority	P3	10	8	6	6
RR (q=2)	P1	10	10	5	0
RR (q=2)	P2	9	8	5	1
RR (q=2)	P3	6	4	2	2

Table 6. Analytical per-process scheduling results

Chapter 7: Comparison Graph and Interpretation

Algorithm	Avg. WT	Avg. TAT	Avg. RT
FCFS	3.33	6.67	3.33
SJF	3.00	6.33	3.00
Priority	3.33	6.67	3.33
RR (q=2)	4.00	7.33	1.00

Table 7. Average performance comparison in logical time units

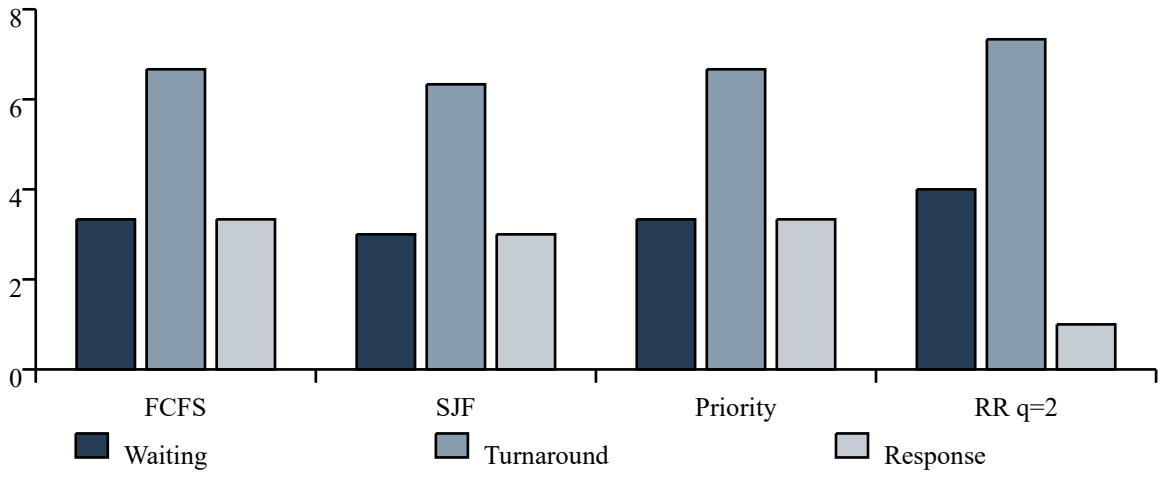


Figure 3. Average scheduling metrics for the example workload

7.4 Interpretation and Trade-offs

SJF gives the lowest average waiting and turnaround times in this example because P3 runs before P2 after P1 finishes. FCFS and Priority coincide because P1 is the only ready process at time zero, and P2 has both the earlier arrival and the higher priority among the remaining jobs.

Round Robin gives the lowest average response time: each process begins execution by time 4. However, P1 and P2 finish later than under FCFS, increasing average waiting and turnaround times. The result demonstrates a trade-off between early service and final completion rather than a universally superior policy.

7.5 Evaluation Limits

All schedules finish at time 10 and have no idle interval. Under the stated zero-overhead model, CPU utilisation over [0,10] is 100% and throughput is 3/10 processes per time unit. These are derived values, not fields reported by the supplied program. Real context-switch costs, I/O and unpredictable CPU bursts would change the model.

Chapter 8: Source Organisation and Reproducibility

8.1 Available Project Material

The project material consists of the Review 1 proposal and the C++ code-and-implementation PDF. The proposal establishes the problem, objectives and planned features. The implementation document describes the console program, includes the scheduling logic and provides a screenshot of process input and algorithm selection.

8.2 Source Organisation

The documented implementation can be organised in one main.cpp file. Its logical order is: standard-library headers; Process, Segment, Metrics and Result definitions; validation; simulation; result printing; and main. This arrangement is sufficient for the small educational program while allowing the engine to be separated into its own module later.

8.3 Reproduction Steps

Use the three-process workload in Table 5. Build the case-correct source with a compatible C++ compiler, select Compare All, and enter quantum 2. Compare the output against Tables 6 and 7. Check the execution intervals before checking averages, because an incorrect order can produce plausible-looking but incorrect metrics.

For an independent manual check of P2 under Round Robin: its first start is 2, completion is 9, arrival is 1 and burst is 3. Thus RT = 2 - 1 = 1, TAT = 9 - 1 = 8 and WT = 8 - 3 = 5. Applying the same method to every row reproduces the average results.

8.4 Repository and Build Status

No repository URL for the Smart CPU Scheduling Simulator is supplied in the attached project material. The repository in the formatting reference belongs to a different project and is not used here. The executable was not provided, and the PDF code was not compiled for this report. A reproducible final submission should include the case-correct source file, build instructions and captured output from the executable.

8.5 Suggested Submission Package

The submission package should contain this report, the source file, the original input workload, and actual output captures for all four algorithms. Recording the compiler version and quantum alongside the test input makes the demonstration repeatable. These items are recommended supporting materials, not claims about files already included.

Chapter 9: Conclusion and Future Scope

9.1 Conclusion

The Smart CPU Scheduling Simulator provides a focused implementation of four standard scheduling algorithms. It accepts arrival, burst and priority data, records a Gantt-style execution timeline and calculates completion, turnaround, waiting and response times. Its comparison option applies all four algorithms to the same workload, helping students explain how selection rules affect process behaviour.

The documented C++ version retains the core scheduling objectives of the original proposal while using a console interface. It makes ready-process selection, idle-time handling, non-preemptive execution and Round Robin queue operations explicit. The worked example shows that SJF can reduce average waiting time while Round Robin can improve the delay before first service.

9.2 Limitations

The model uses a single CPU and one known CPU burst per process. It excludes I/O blocking, context-switch duration, dynamic priority changes, multiple cores and real-time deadlines. It prints results after simulation rather than providing interactive animation. A finite set of prepared inputs cannot establish that one algorithm is best for every workload.

9.3 Future Enhancements

Graphical interface: add process-entry controls, coloured Gantt charts and a consolidated comparison view.
Additional algorithms: include SRTF and preemptive Priority with clear tie-breaking conventions.
Aging: adjust waiting-process priorities to study starvation mitigation.
More realistic modelling: include context-switch overhead, I/O bursts and multiple CPUs.
Export and repeatability: save workloads and results using CSV or JSON.
Automated verification: add deterministic tests for arrival boundaries, idle gaps and queue ordering.

9.4 Learning Outcomes

The project connects Operating Systems theory with C++ structures, vectors, queues, sorting and exception handling. It also demonstrates the importance of consistent input assumptions, transparent tie-breaking and independent result verification when building algorithm simulators.

References

[1] *PROJECT PROPOSAL - REVIEW 1: Smart CPU Scheduling Simulator.* Supplied project document, filename: Review1 os1.pdf. Team: Abhishek Kumar (24BCE2876) and Ashish Ranjan (24BCE2154). Used for project title, team details, motivation, scope and original proposal.

[2] *SMART CPU SCHEDULING SIMULATOR: C++ Code and Implementation.* Supplied project document, filename: Smart_CPU_Scheduling_Simulator_Code_Implementation review 2333.pdf. Used for implemented algorithms, data structures, validation rules, queue ordering and the reproduced console screenshot.

[3] *Real-Time OS Algorithm Visualizer: CPU Scheduling Algorithms & Deadlock Avoidance Simulation.* Supplied reference report by Aryan Dabas, Kritagy Kaushik, Arnav Ray and Anshu Kumar. Used as the structural and visual formatting reference only.

Source and Evidence Note

Project-specific claims in this report are grounded in the supplied proposal and implementation document. The reference report provides the chapter organisation and academic presentation style; its web application, Banker algorithm, repository and performance claims do not describe this project.

The numerical workload in Chapter 7 is taken from the supplied console screenshot. Its schedules, tables and chart are calculated using the scheduling rules visible in the implementation document. The screenshot is reproduced as supplied evidence. No unprovided execution screenshot or benchmark result has been fabricated.

Appendix A: Algorithm Pseudocode

A.1 Non-preemptive Scheduling

```
time = 0; completed = 0
while completed < n:
```

```
    ready = unfinished processes with arrival <= time
if ready is empty:
        next = minimum arrival among unfinished jobs
append IDLE interval [time, next]         time =
next         continue     choose by:         FCFS:
(arrival, PID)
        SJF:      (burst, arrival, PID)
Priority: (priority, arrival, PID)
firstStart[chosen] = time
    append execution [time, time + burst[chosen]]
time  +=  burst[chosen]       completion[chosen]  =
time     mark chosen finished; completed += 1
```

A.2

Round Robin

```
sort process indices by (arrival, PID) initialise
remaining bursts and empty FIFO queue while completed
< n:     if queue empty and next arrival is in the
future:
        append IDLE interval; advance time
enqueue all unqueued arrivals <= time     take
process i from queue front     if i has never
run: firstStart[i] = time     duration =
min(quantum, remaining[i])     append
execution [time, time + duration]     time +=
duration; remaining[i] -= duration     enqueue
newly arrived processes <= time     if
remaining[i] > 0: enqueue i at the back
else: completion[i] = time; completed += 1
```

A.3 Metric Computation

For every process, calculate TAT = CT - AT, WT = TAT - BT and RT = firstStart - AT. Add the individual values and divide each total by the process count to obtain its average. Print the timeline and metrics together so the arithmetic can be checked against the execution order.

Appendix B: Additional Test Cases and Expected Outputs

These cases provide small, manually checkable reference outputs. They should be executed against the final source before marking runtime testing complete. All times are logical units and overhead is zero.

Case	Workload / policy	Expected timeline and checks
B1: Initial idle	P1: AT=3, BT=2, priority=1. FCFS.	IDLE [0-3], P1 [3-5]. CT=5, TAT=2, WT=0, RT=0.
B2: Idle gap	P1: AT=0, BT=1; P2: AT=4, BT=2. FCFS.	P1 [0-1], IDLE [1-4], P2 [4-6]. Both WT and RT are zero.

B3: SJF selection	P1: AT=0, BT=6; P2: AT=0, BT=2; P3: AT=0, BT=3.	P2 [0-2], P3 [2-5], P1 [5-11]. Average WT=7/3; average TAT=6.
B4: RR boundary	P1: AT=0, BT=4; P2: AT=2, BT=1; q=2.	P1 [0-2], P2 [2-3], P1 [3-5]. P2 RT=0; P1 WT=1.
B5: Priority tie	P1: AT=0, BT=2, priority=1; P2: AT=0, BT=1, priority=1.	P1 [0-2], P2 [2-3], because PID breaks the tie.
B6: Large quantum	P1: AT=0, BT=3; P2: AT=1, BT=2; q=10.	P1 [0-3], P2 [3-5]. Matches FCFS for this workload.
B7: Invalid burst	Any process with BT=0.	Reject: burst time must be greater than zero.
B8: Invalid quantum	Select RR and enter q=0.	Reject: quantum must be a positive integer.

Table 8. Additional test cases and expected outputs

B.1 Suggested Verification Record

For each case, retain the entered workload, selected algorithm, quantum when applicable, actual timeline, calculated metrics and pass/fail comparison. A case passes only when both its execution order and metric values match the expected output. Keep compiler and source-version details with the test record.